

我為什麼開始研究 AST Active Healer ?

個人技術核心 | AST Active Healer 與程式生成可靠性研究

01 研究起點 | 當雲端生成成本開始成為系統問題

出題系統每次生成程式都需呼叫雲端 AI，API 成本隨使用逐漸累積。我嘗試把部分工作移到本地小模型，但實測後成功率並不理想，且常出現重複性的格式、介面與資料流錯誤。

研究問題 怎麼讓小模型少做一點，卻做得更可靠？

我觀察到，同一數學單元的程式結構與常見錯誤會反覆出現，因此把問題拆成兩個可驗證的子問題：

Q1 哪些重複工作可以事先抽離？

Q2 哪些局部錯誤可用固定規則安全修復？

02 從問題逐步長出的研究設計

01 | Scaffolding Prompt

先把重複工作移出模型

預先提供題型規則與函數庫。

02 | AST Active Healer

只修能確定的局部錯誤

以固定規則修復局部錯誤。

03 | MCRI | 自建評估指標

Math Code Reliability Index

初期的生成品質評估框架

後續研究再重新檢討其限制。

Scaffolding 負責降低生成前的規劃負擔；AST Active Healer (以下簡稱 Active Healer / Healer) 負責處理生成後可明確判定的局部錯誤。

03 初版驗證 | 我第一次如何測試這個想法？

本研究以可重複執行的 Python 程式作為生成單位；每支程式可反覆產生同型但數值不同的變體題與標準答案。初版以 Gemini 3.0 Flash、Qwen 3.0 14B、Qwen 3.0 8B 交叉三種策略，共形成 9 組條件；產出 405 支 Python 程式、2,025 道數學題，並以自建指標 MCRI (0-100) 進行初步評估。

策略	內容	目的
Ab1 Bare	僅提供基礎任務指令	基礎對照
Ab2 Scaffolding	加入鷹架提示與 Domain Functions	降低模型規劃負擔
Ab3 Full Healer	在 Ab2 上加入 Active Healer	修復可判定的局部錯誤

三組逐步加入元件，比較 Ab1 → Ab2 → Ab3 的差異，即可分別看出鷹架與 Healer 的增量效果。

Scaffolding 如何進入生成流程



初版 Active Healer 怎麼運作？

以及我當時如何評估它

01 Active Healer 的設計構想

即使加入 Scaffolding，小模型生成的 Python 程式仍常出現格式、語法與結構錯誤。因此我設計 Active Healer，嘗試修復生成程式中反覆出現的局部問題。

02 初版四層修復機制

初版 Active Healer

層級	修復器名稱	處理問題
1	RegexHealer	模型說明文字殘留、Markdown、明確格式問題
2	ASTHealer	縮排、return、語法與結構問題
3	AntiDuplicationHealer	重複宣告、賦值順序、變數遮蔽
4	UnifiedCleanupHealer	未使用變數、部分執行問題與統一清理

03 初版評估方式：自創指標 MCRI (Math Code Reliability Index)

Program Value	50%	Math Value	50%
工程基石與資料衛生	30%	數學正確性	20%
程式架構品質	20%	教學有效性	30%

初版研究同時評估「程式本身的可靠性」與「生成數學內容的品質」。

後續旺宏決賽階段，我重新檢討 MCRI 是否足以作為核心效果證據。

本研究關鍵術語 後續頁面沿用

名詞	說明
AST	抽象語法樹 Abstract Syntax Tree 以樹狀結構表示程式語法，方便定位與修改程式結構。
Active Healer	本研究設計的固定規則程式修復機制。
Scaffolding	在生成前先提供題型規則、工具與結構，降低模型自行規劃負擔。
Domain Functions	預先建立、可供模型直接呼叫的數學工具函數。
MCRI	Math Code Reliability Index 初版研究使用的自建程式與數學品質評估指標。
消融實驗	Ablation Study 逐步加入或移除元件，比較各方法對結果的增量影響。

初版實驗結果

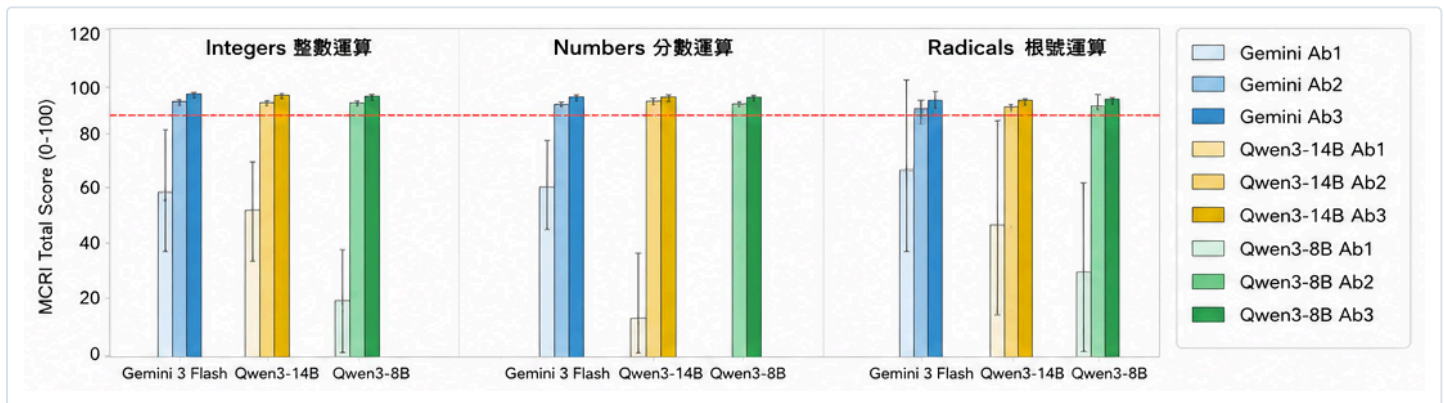
為什麼當時我會認為這套方法有效？

01 初版實驗怎麼跑的？

01	課本例題	整數 / 分數 / 根號
02	建立鷹架	Prompt + Domain Functions
03	AI 生成	不同難度 Python 程式
04	Active Healer	四層固定規則修復
05	MCRI 評分	程式可靠性 + 數學品質

從課本例題抽取題型結構、建立生成鷹架，再由模型產生程式、Healer 修復，最後以 MCRI 評估。

02 三個運算領域的 MCRI 結果



三種模型 × 三種策略在整數、分數、根號運算的 MCRI 總分；紅色虛線為 90 分門檻。

圖表解讀

在 Ab1 下，模型間差異很大，較小的 Qwen 模型尤其不穩定；加入 Scaffolding 後，三種模型在三個運算領域的 MCRI 都明顯提高、差距縮小；再加入 Active Healer 後，平均分數進一步提升，結果更集中。

03 初版結果摘要

整體平均 MCRI

Ab1 Bare **41.00** → Ab2 Scaffolding **94.39** → Ab3 Full Healer **97.41**

跨 3 模型 × 3 運算領域

標準差

34.03 → 1.96 → 1.67

這些跨模型、跨運算領域的一致趨勢，使我當時認為方法已有效改善程式生成的不穩定問題。

Kruskal-Wallis : $p < .001$; Ab3 vs. Ab2 : $d = 1.66$

當時，我以為這些結果已經足以支持方法有效。

97.41 是怎麼得到的？

重新檢查初版驗證，以及旺宏評審的追問

97.41 為三種模型與三個運算領域 Ab3 結果的整體平均，並非每個模型各自的固定表現。

01 沿著初版流程，我看見三個驗證缺口



沿著原本得到 97.41 的流程重新檢查，我發現初版驗證本身存在三個方法學缺口：

① 測試資料真的獨立嗎？

題型、規則與 Healer 修復方式在實驗過程中持續整理形成，需要確認正式測試是否真正獨立於方法開發。

② Healer 的增量效果

Ab2 → Ab3 : 94.39 → 97.41 (+3.02)

這 +3.02 來自哪些錯誤被修復？哪些錯誤其實修不了？

③ 評估獨立性

評估使用自建的 MCRI。高分可以描述結果，但是否足以作為核心證據，仍需更獨立、直接的驗證。

02 旺宏評審進一步追問的核心問題

我重新檢查時發現的問題，與旺宏決賽評審的追問高度重疊；評審的焦點在原創貢獻、獨立驗證、公平比較與 Healer 的實際修復能力。

► [初版創意說明書](#) ↗ ► [評審建議原文](#) ↗

01 聚焦真正的原創貢獻

系統整合很多，但真正需要做深的是 Active Healer。

02 建立更有說服力的獨立驗證

不能只依賴自建評估指標，需要更直接的效果驗證。

03 建立公平比較條件

正式比較前應固定 Prompt 與修復規則，避免方法與評估資料互相影響。

04 把 Healer 的增量效果拆開看

不能只看總分上升，而要逐筆確認哪些原始錯誤真的被修復。

03 評審意見如何轉成下一版實驗

初版驗證缺口	下一版實驗修改
原創貢獻分散	→ 決賽驗證聚焦 Active Healer
方法開發與測試資料未充分分離	→ 規則開發與保留測試分離
實驗過程仍可能持續調整條件	→ 正式評估前凍結提示條件與修復規則
MCRI 無法直接回答「到底修了什麼」	→ 逐筆追蹤 原始失敗 → 修正後 → 通過 / 失敗

97.41 的結果沒有改變，但我開始重新檢視這套驗證是否足以支持方法有效。

初版結果值得繼續研究，但還不足以回答 Active Healer 在獨立資料上真正能修復多少錯誤。

重新驗證：我怎麼把開發與正式測試分開？

[▶ 決賽成果報告書 ↗](#)

重新限制題目來源、Prompt、Healer 規則與評估方式。

01 重新定義比較條件

決賽重新使用 Math16 題庫：16 道國中會考數學題，涵蓋 4 個數學領域，並比較 3 種模型與 4 種 Prompt 條件。

條件	提供給模型的資訊	比較重點
Ab1	僅原始題目	基礎對照
Ab2g	原始題目 + 一般程式規格	一般規格是否有幫助
Ab2d+api	原始題目 + Domain API 鷹架	領域工具帶來多少幫助
Ab2d+spec	原始題目 + 題目專屬規格鷹架	題型規格帶來多少幫助

註：Ab2d+api 與 Ab2d+spec 使用既有通用 Domain API，未針對 Math16 各題新增 API。

02 規則開發與正式測試分離

16 題 × 3 種模型 × 4 種 Prompt 條件 × 5 組固定隨機種子 = 960 個實驗單元

4 題 規則開發		12 題 正式測試
實驗單元	240	720
用途	設計 Prompt、分析錯誤、建立 Healer 規則	驗證固定後的 Prompt 與 Healer 規則
規則狀態	可依開發結果調整	全部凍結
統計用途	不納入主要成效	核心研究主要評估資料

規則開發 → 規則凍結 → 正式測試

同一題的所有結果只放在其中一組，避免同一題同時用於規則開發與正式測試。

03 正式評估怎麼判定 Healer 是否真的修好



不看答案

Healer 不讀取正確答案。

規則凍結

正式測試開始後，不再修改 Prompt 或 Healer 規則。

安全介入

只有修法可由固定規格明確判定時才修改；否則不介入。

正式測試下，Healer 真正能安全修復多少？

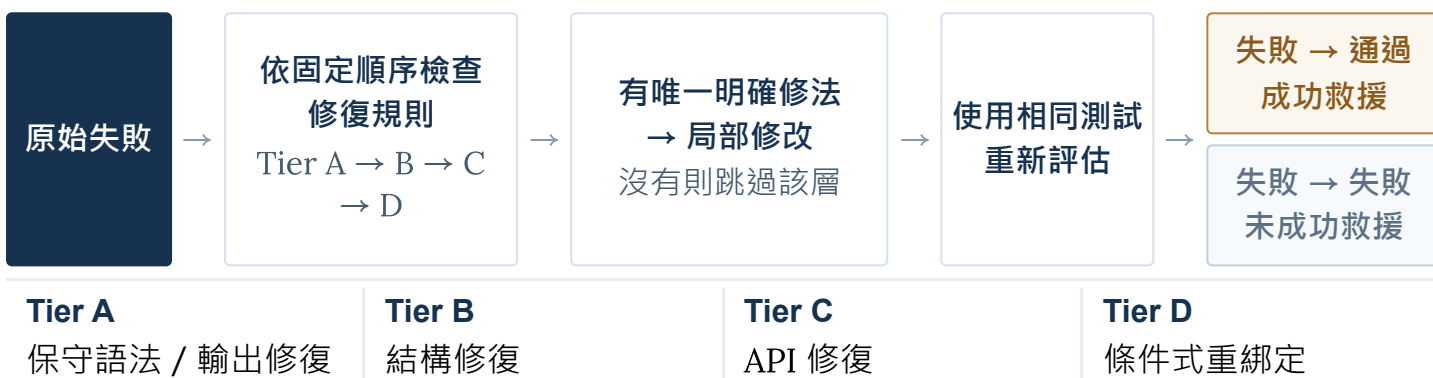
能修什麼、怎麼修、正式測試下救回多少

01 Healer 的能力邊界

可以修	不由 Healer 修
· 介面與輸出格式 · 局部程式結構 · 資料流與組裝問題 · 可由固定 API 規格唯一判定的局部錯誤	· 數學答案錯誤 · 核心演算法錯誤 · 題意理解錯誤 · 需要重新推理或重新解題的問題

Healer 不讀取正確答案；只有在固定規格能唯一決定修法時才介入，否則不修改。

02 正式測試下的救援判定流程



03 正式測試結果：真正救回多少？



安全救援率 = $9 \div 342 = 2.63\%$

42 個實際修改案例中，9 個轉為通過、33 個仍失敗；修改後成功比例為 21.43%。

04 為什麼只有 2.63%？看失敗類型

失敗類型	原始失敗	成功救援
程式解析 / 語法	114	1
格式 / 入口 / 輸出包裝	22	6
Domain API / 工具使用	3	0
執行 / 組裝 / 資料流	59	2
答案 / 演算法錯誤	144	0

342 個原始失敗中，144 個（約 42%）屬於答案或演算法錯誤，需要重新理解題意或改寫演算法，超出 Healer 的介入範圍；9 個成功救援主要來自介面、格式與資料流等局部問題。

重新驗證讓我看見：Healer 能可靠處理的，是修法可由固定規則唯一判定的局部錯誤；大量失敗其實發生在更前面的答案、演算法或生成規劃階段。

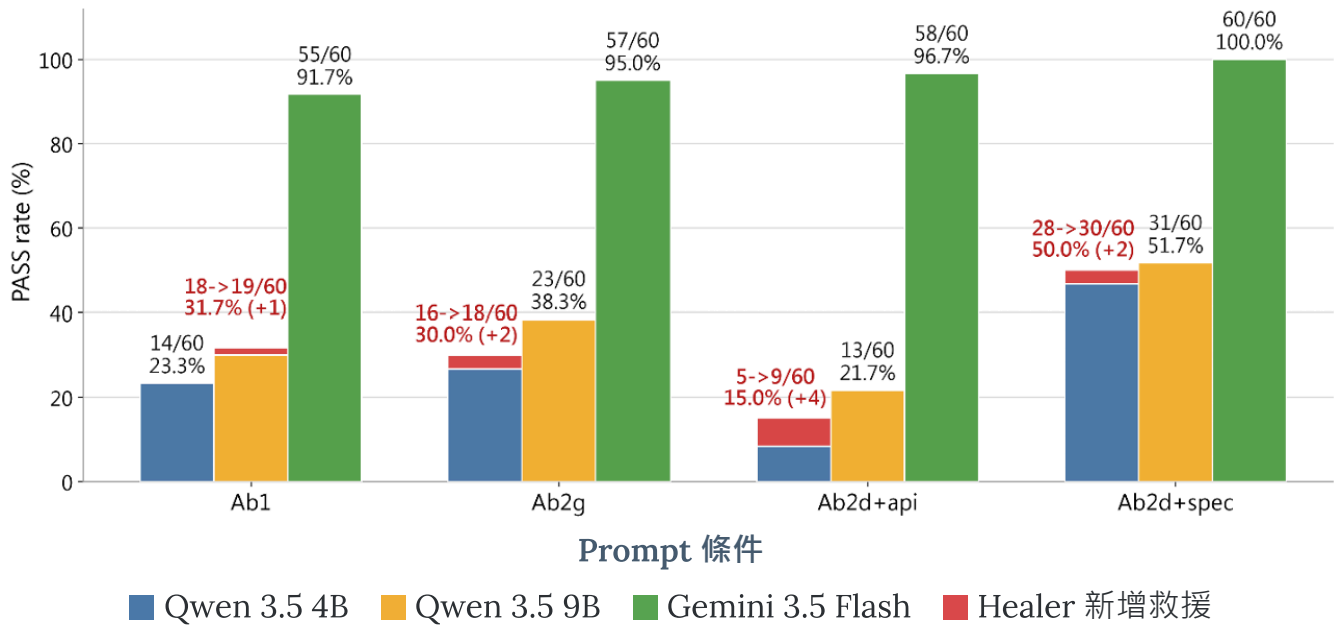
因此下一步改問：如果補足模型需要的領域工具與流程資訊，前端生成失敗是否會下降？

模型與提示條件造成多大的差異？

正式測試結果、新的研究問題與延伸驗證設計

01 正式測試主結果

三種模型在四種 Prompt 條件下的原始通過率與 Healer 新增救援



模型差異

Gemini 3.5 Flash 在四種 Prompt 條件下皆維持較高原始通過率。

提示條件差異

兩個 Qwen 模型皆在 Ab2d+spec 表現最好、Ab2d+api 最差；提供更多工具資訊本身不一定能改善生成。

Healer 增量有限

342 格原始失敗 中只有 9 格被安全救回，主要差異仍來自模型與 Prompt 條件。

02 正式結果帶出的下一個問題

核心研究使用先前依課本題型建立的通用 Domain API，並未針對 Math16 各題補齊工具，因此進一步檢查「工具覆蓋」與「工具使用流程」兩個因素。

03 延伸驗證：補足工具與流程資訊

原有 Domain API
依原本課本題型建立

Math16 工具缺口
部分題型缺少所需工具

補齊 Domain Functions
涵蓋 Math16 全部 16 題

A | API 選單提示

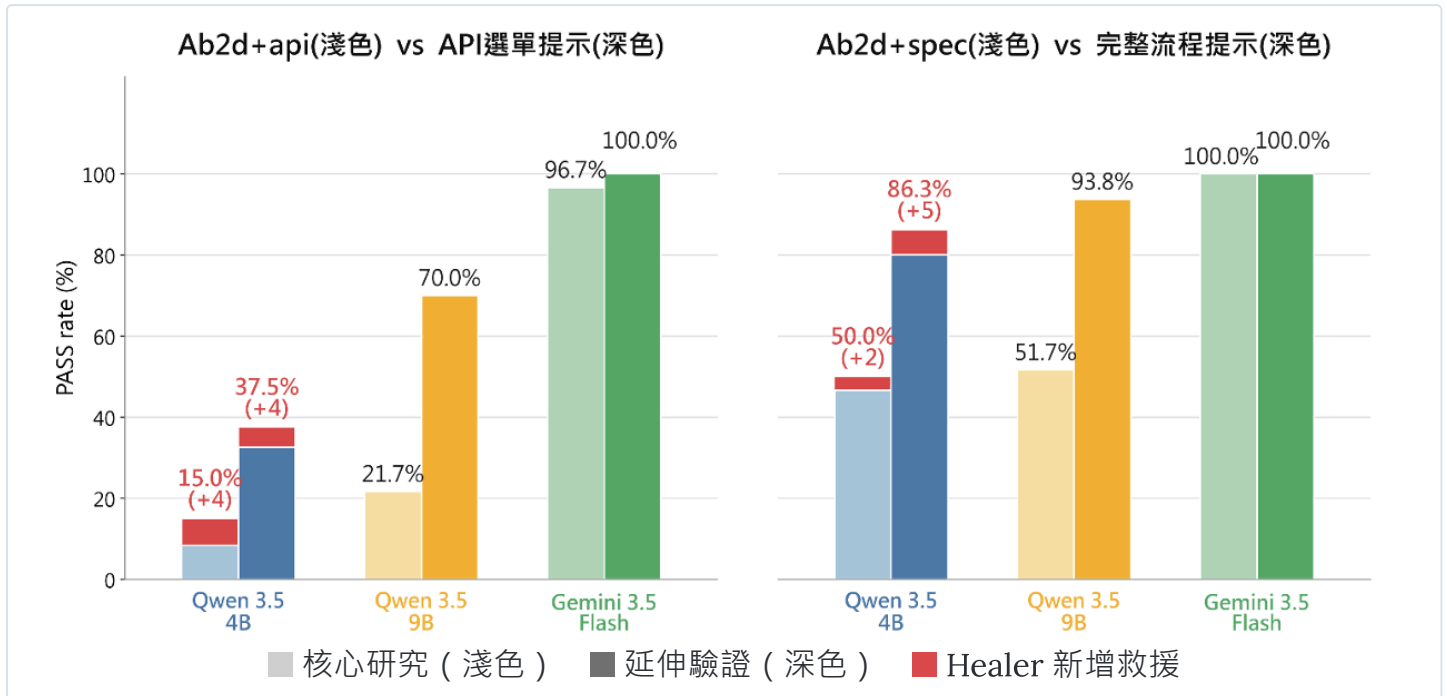
提供完整 API 規格與呼叫方式，由模型自行選擇所需工具並規劃處理流程。

B | 完整流程提示

使用相同工具，進一步明確指定必要 API、參數來源、呼叫順序、中間資料流與答案組裝方式。

這組延伸驗證另行進行，不納入前述正式測試結果；目的在比較補齊領域工具後，工具資訊與流程鷹架各自對程式生成表現的影響。

補足工具與流程資訊後，生成可靠性有沒有改善？



01 結果解讀

工具覆蓋

補齊 Domain Functions 後，兩個 Qwen 模型的原始通過率明顯提高；Gemini 原本已接近滿分，變化有限。

流程鷹架

只提供工具清單仍不足；加入完整流程資訊，可進一步改善部分模型。

Healer 的角色沒有改變

新增救援仍集中於可由固定規格判定的局部錯誤。

02 延伸驗證得到的設計分工

元件	在本研究中的作用
Domain Functions	提供模型完成題目所需的領域工具
完整流程提示	降低工具選擇、參數來源、呼叫順序與資料流規劃負擔
Active Healer	處理修法可由固定規則唯一判定的局部錯誤

延伸驗證顯示，工具覆蓋與流程規劃都會影響中小型模型的程式生成表現；僅提供 API 並不一定足夠。本研究結果支持上述分工，但實際部署效益仍需後續驗證。

03 決賽評審追問

「如果 Syntax Error 可以直接交回大型語言模型重新修復，為什麼還需要 AST Healer？」

直接把錯誤交回模型重新生成並不保證能修好，尤其小模型可能再次犯錯，也可能改壞原本正確的部分；重新生成本身也較難控制與追蹤修改內容。Healer 因此只處理修法可由固定規則唯一判定的局部錯誤，以可追蹤、可重現的方式修改；無法確定時就不介入。

研究反思

初版的 97.41 一度讓我以為問題已經解決；重新驗證得到 2.63% 後，我才開始區分「結果變好」與「方法被充分驗證」。決賽與後續實驗讓我更清楚 Healer 的適用範圍，也讓研究問題從「能不能做出來」，轉向「在什麼條件下有效、為什麼有效」。